

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

foodmood

Ingegneria del Software e Progettazione Web
2025-2026

Gabriele Monti

SOMMARIO

1. SPECIFICHE DEI REQUISITI SOFTWARE.....	3
1.1 INTRODUZIONE.....	3
1.1.1 SCOPO DEL DOCUMENTO.....	3
1.1.2 PANORAMICA DEL SISTEMA	3
1.1.3 REQUISITI HARDWARE E SOFTWARE.....	4
1.1.4 SISTEMI CORRELATI	4
1.2 USER STORIES	5
1.3 REQUISITI FUNZIONALI	5
1.4 USE CASE.....	6
1.4.1 DIAGRAMMA DEI CASI D'USO	6
1.4.2 INTERNAL STEPS.....	7
2. STORYBOARDS.....	8
3. DESIGN.....	11
3.1 CLASS DIAGRAM	11
3.1.1 VOPC.....	11
3.1.2 DESIGN LEVEL.....	12
3.2 DESIGN PATTERNS.....	13
3.2.1 ABSTRACT FACTORY.....	13
3.2.2 SINGLETON.....	13
3.2.3 DECORATOR	14
3.2.4 FACTORY	14
3.3 ACTIVITY DIAGRAM	15
3.4 SEQUENCE DIAGRAM	17
3.5 STATE DIAGRAM.....	17
4. TESTING	18
5. CODE.....	18
5.1 GITHUB.....	18
5.2 SONARCLOUD	18
6. VIDEO.....	18
6.1 YOUTUBE.....	18

1. SPECIFICHE DEI REQUISITI SOFTWARE

1.1 INTRODUZIONE

1.1.1 SCOPO DEL DOCUMENTO

Il presente documento descrive i requisiti funzionali e non funzionali del sistema FoodMood, sviluppato nell'ambito del corso di Ingegneria del Software e Progettazione Web, anno accademico 2025/2026.

Il documento fornisce una panoramica degli obiettivi del sistema, delle funzionalità principali e dei requisiti tecnici, al fine di guidare le attività di analisi, progettazione e sviluppo e di definire in modo chiaro ciò che il sistema deve offrire ai suoi utilizzatori.

1.1.2 PANORAMICA DEL SISTEMA

FoodMood è un sistema software finalizzato a supportare i principali attori del settore della ristorazione (clienti, titolari e camerieri) con l'obiettivo di semplificare e migliorare l'esperienza di fruizione e gestione del servizio.

Funzionalità lato cliente

Il sistema consente al cliente di accedere alle funzionalità in modalità ospite oppure tramite registrazione. La registrazione abilita funzionalità aggiuntive, quali il salvataggio delle preferenze alimentari e l'accesso a eventuali servizi di fidelizzazione, ove previsti dal ristorante.

Il sistema è concepito come servizio multi-ristorante. Ciascun ristorante può essere configurato in modo indipendente, al fine di adattare il sistema alle specifiche esigenze operative e organizzative del singolo esercizio.

Il sistema rende disponibile una modalità di ordinazione guidata che consente al cliente di inserire preferenze e di ottenere proposte di piatti coerenti, generate a partire dalle informazioni presenti nel menù digitale.

Il sistema supporta inoltre le funzionalità tipiche di un servizio ristorativo digitale, includendo la consultazione del menù digitale e l'ordinazione manuale tramite selezione dei piatti.

Funzionalità lato titolare

Il sistema consente al titolare di gestire il menù digitale e i relativi contenuti informativi.

Il titolare può definire e mantenere l'elenco degli ingredienti, associando a ciascuno macronutrienti e allergeni.

Può inoltre creare e modificare i piatti del menù specificando ingredienti associati, prezzo, disponibilità, descrizione e un'immagine, così da supportare una scelta consapevole da parte del cliente.

Funzionalità lato cameriere

Il sistema consente ai camerieri di registrare e gestire le comande e di ricevere notifiche relative a richieste provenienti dai tavoli (ad esempio chiamata da parte di un cliente e richiesta del conto), al fine di supportare un intervento tempestivo durante il servizio.

1.1.3 REQUISITI HARDWARE E SOFTWARE

Requisiti Hardware

Il sistema può essere eseguito su un normale computer in grado di eseguire applicazioni Java. Sono richiesti almeno 512 MB di RAM e 512 MB di spazio su disco. Il sistema è compatibile con i principali sistemi operativi, tra cui, macOS, Linux e Windows.

Requisiti Software

Il sistema è sviluppato in Java e richiede un JDK compatibile con Java 23. La gestione del ciclo di build, delle dipendenze e dell'esecuzione in ambiente di sviluppo è basata su **Apache Maven**, di conseguenza è richiesta la disponibilità di Maven (installato localmente oppure integrato nell'IDE).

Lo sviluppo e la manutenzione del progetto possono essere effettuati tramite **IntelliJ IDEA** oppure **Visual Studio Code**. L'interfaccia grafica è realizzata mediante **JavaFX** e file **FXML**; per la progettazione e la modifica dei file FXML è richiesto **JavaFX Scene Builder**.

La persistenza dei dati richiede un **DBMS MySQL**, la connessione al database è gestita tramite **MySQL Connector/J** (dipendenza `mysql-connector-j`, versione **9.2.0**), risolta automaticamente da Maven.

Il controllo qualità del codice sorgente è effettuato mediante **SonarCloud**.

Per la modellazione e la documentazione dei diagrammi UML è stato utilizzato **StarUML**.

1.1.4 SISTEMI CORRELATI

Sistema 1: Pienissimo Pro

- **Pro:** Offre strumenti completi per la gestione operativa del locale (ad esempio prenotazioni e organizzazione del servizio) e supporta la digitalizzazione del menù e funzionalità basate su QR code.
- **Contro:** La selezione dei piatti richiede maggiore iniziativa da parte del cliente, che deve consultare il menù per individuare le opzioni più adatte alle proprie preferenze, non è possibile poi consultare i dati nutrizionali di ciascun piatto.

Sistema 2: TheFork Manager

- **Pro:** Fornisce un gestionale strutturato per ristoranti, utile per gestire prenotazioni, sala e relazione con il cliente. Consente di rendere disponibile il menù in formato digitale tramite QR code per facilitarne la consultazione.
- **Contro:** L'esperienza lato cliente è prevalentemente di consultazione: per trovare i piatti più adatti alle proprie esigenze il cliente deve svolgere sempre una ricerca dei piatti su tutto il menù. Il sistema non è progettato specificamente per proporre scelte personalizzate a partire da preferenze e informazioni nutrizionali.

FoodMood si distingue in quanto supporta il cliente nell'ordinazione in base alle proprie preferenze e consente al ristoratore di inserire e mantenere informazioni nutrizionali accurate sui piatti.

1.2 USER STORIES

1. User story

Come cliente del ristorante,

voglio ricevere suggerimenti di piatti basati sulle mie preferenze dietetiche e allergie,

in modo da trovare opzioni compatibili con le mie esigenze.

2. User story

Come titolare del ristorante,

voglio inserire i piatti nel menù digitale specificando ingredienti, allergeni e valori nutrizionali,

in modo da fornire ai clienti informazioni sui piatti scelti.

3. User story

Come cameriere,

voglio visualizzare la mappa della sala con lo stato aggiornato di ciascun tavolo,

in modo da coordinare l'accoglienza e il servizio.

1.3 REQUISITI FUNZIONALI

1. Requisito funzionale

Il **sistema deve fornire** al cliente, prima della conferma dell'ordine, un riepilogo con i piatti selezionati, la quantità per ciascun piatto e il subtotale*.

2. Requisito funzionale

Il **sistema deve mostrare** al cliente, durante la visualizzazione del menù digitale, esclusivamente i piatti appartenenti alla tipologia di portata* selezionata.

3. Requisito funzionale

Il **sistema deve mostrare** al cameriere la mappa della sala con lo stato* di ciascun tavolo.

GLOSSARIO

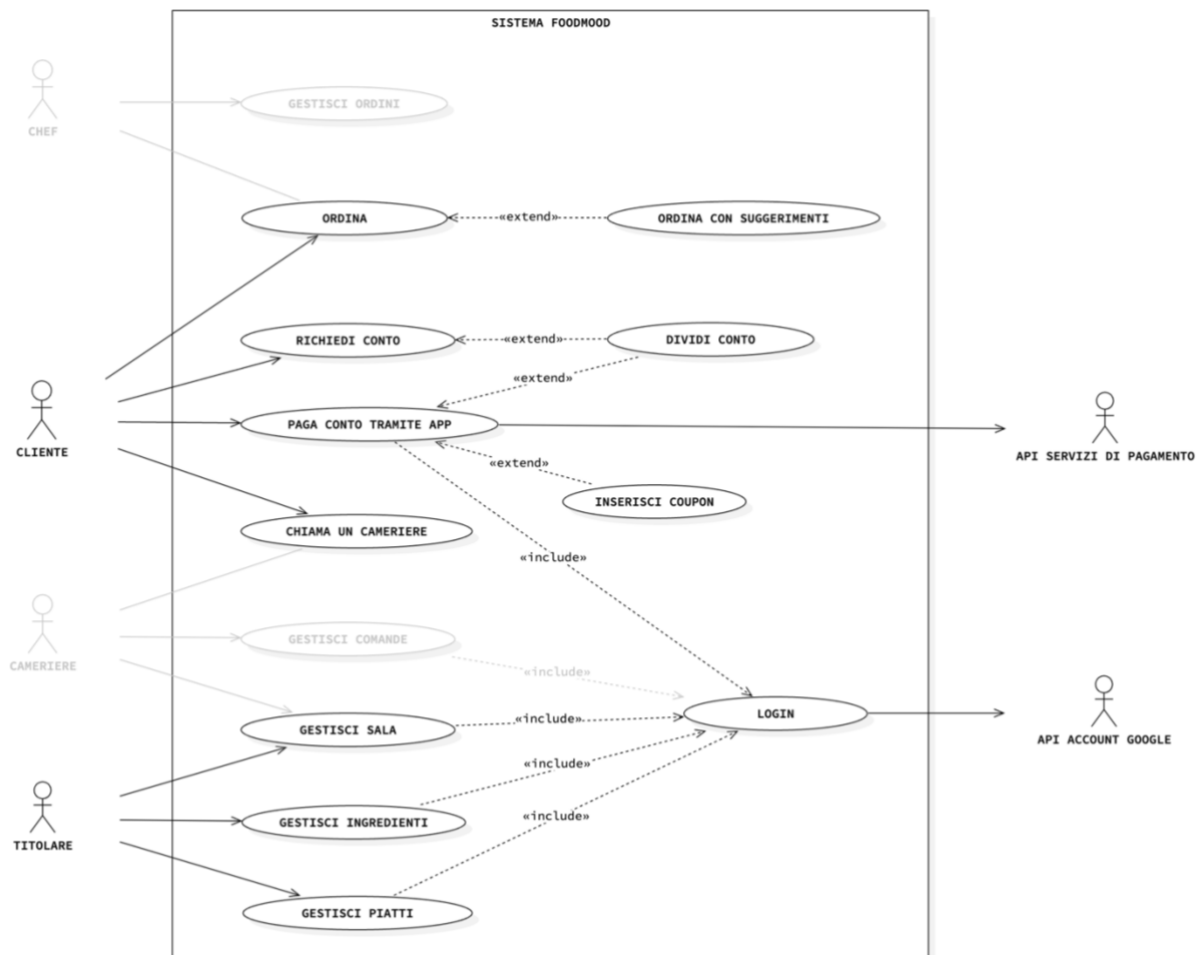
Subtotale: importo parziale dell'ordine, calcolato come somma dei prezzi degli articoli selezionati moltiplicati per la quantità.

Tipologia di portata: categoria del menù associata a ciascun piatto.

Stato: valore associato a un tavolo nella mappa della sala, i valori ammessi sono: libero, prenotato, occupato.

1.4 USE CASE

1.4.1 DIAGRAMMA DEI CASI D'USO



NOTE

Il diagramma non riflette appieno quanto implementato: i casi d'uso "Paga conto tramite app", "Gestisci comande", "Gestisci ordini" non sono stati implementati nel codice.

1.4.2 INTERNAL STEPS

Caso d'uso: ordina con suggerimenti

1. Il cliente sceglie di ordinare ricevendo suggerimenti.
2. Il sistema mostra al cliente le tipologie di portata* selezionabili.
3. Il cliente seleziona una o più tipologie di portata e conferma.
4. Il sistema determina la complessità dell'ordine* in base al numero e alla tipologia di portate selezionate.
5. Il sistema mostra al cliente ulteriori preferenze da selezionare in base alla complessità determinata.
6. Il cliente seleziona le preferenze richieste* e conferma.
7. Il sistema filtra i piatti disponibili in base ai criteri forniti e mostra una lista di piatti suggeriti*.
8. Il cliente aggiunge all'ordine uno o più piatti proposti, indicando la quantità per ciascun piatto.
9. Il sistema mostra il riepilogo con piatti selezionati, quantità e sottotale prima della conferma.
10. Il cliente conferma l'ordine finale.
11. Il sistema registra l'ordine e mostra la conferma di successo.

Estensioni

- 3a. **Il cliente non seleziona alcuna portata.**
 - 3a.1 Il sistema informa che è necessario selezionare almeno una portata. **Torna allo step 3.**
- 4a. **Il sistema determina che l'ordine è semplice e non richiede ulteriori preferenze. Vai allo step 7.**
- 7a. **Il sistema non trova piatti compatibili con i criteri forniti e informa il cliente.**
 - 7a.1 Il cliente ricomincia la procedura. **Torna allo step 2.**
 - 7a.2 Il cliente annulla l'operazione. **Il caso d'uso termina.**

GLOSSARIO

Tipologie di portata: categoria del menù associata a ciascun piatto. Antipasto, primo, secondo, pizza, contorno, frutta, dolce e bevanda.

Complessità dell'ordine: classificazione usata dal sistema per decidere quali preferenze richiedere al cliente in base alle portate selezionate.

Preferenze richieste: insieme di preferenze che il sistema può richiedere. Preferenze dietetiche, allergeni o intolleranze, apporto calorico e budget da rispettare.

Piatti suggeriti: piatti del menù che risultano compatibili con i criteri inseriti dall'utente.

2. STORYBOARDS

In questa sezione vengono mostrate sotto forma di storyboards alcune funzionalità del sistema FoodMood.

CLIENTE



ORDINA

Ordina su misura per te

Cosa vorresti ordinare?

☐ Antipasto

☒ Pranzo

☐ Secondo

☐ Pizza

☐ Contorno

☐ Frutta

☐ Bevanda

☐ Dolce

ORDINA

Ordina su misura per te

Segui un particolare stile alimentare?

☒ Tradizionale

☐ Vegano

☐ Vegetariano

☐ Senza Glutine

☐ Senza Lattosio

ORDINA

Ordina su misura per te

Hai allergie o intolleranze alimentari?

☐ Anidride solforosa

☐ Uova

☐ Latta e derivati

☐ Soia

☐ Glutine

☐ Sesamo

☐ Frutta a guscio

☐ Senape

☐ Sedano

☐ Crostacei

☐ Arachidi

☐ Pesce

☐ Lupini

☐ Molluschi

ORDINA

Ordina su misura per te

Vuoi che il tuo pasto rientri in un certo apporto calorico?

☒ Leggero (fino a 650 Kcal)

☐ Moderato (fino a 1200 Kcal)

☐ Abbondante (fino a 1600 Kcal)

☐ Nessun limite

ORDINA

Ordina su misura per te

Vuoi rimanere entro un certo budget per il tuo pasto?

☒ Economico (entro i 40 €)

☐ Equilibrato (entro i 60 €)

☐ Premium (entro i 90 €)

☐ Nessun limite

ORDINA

Ordina su misura per te

Abbiamo pensato di proporti:

Spaghetti carbonara

10,00 €

Aggiungi all'ordine

Tagliata di manzo

18,00 €

Aggiungi all'ordine

Piatto di ferro

4,00 €

Aggiungi all'ordine

Coca cola

3,00 €

Aggiungi all'ordine

ORDINA

Ordine

Articolo	Quantità	Prezzo	
Spaghetti carbonara	1	10,00 €	
Tagliata di manzo	1	18,00 €	
Piatto di ferro	1	4,00 €	
Coca cola	1	3,00 €	

Totale: 35,00 €

9



Ristorante il Casale

Manager Ristorante

GESTIONE INGREDIENTI

Ricerca:

Cerca ingredienti per nome

Nome	Unità	Scatole	Cant'Inventario	Alloggio
ACETO BALSAMICO	litri	75,0	0,5/170,0/0	ALLERGENO NON PRESENTI
ACQUA	litri	0,0	0,0/0,0/0,0	ALLERGENO NON PRESENTI
AGLIO	kg	160,0	0,4/0,0/0,0	ALLERGENO NON PRESENTI
AGLIO POPOLINE	g	300,0	16,8/17,0/17	ALLERGENO NON PRESENTI
ANCHOVE	g	615,4	25,8/161,49/2	ANCHORE
ANIMON	g	510,0	0,8/110,0/1	ALLERGENO NON PRESENTI
ANANA	g	99,1	11,0/0,0/0,0	ALLERGENO NON PRESENTI
ANACARDIO	g	29,1	3,2/3,2/0,1	ALLERGENO NON PRESENTI
ANANA	kg	75,0	0,4/0,0/0,0	ALLERGENO NON PRESENTI
ANZICO	g	144,0	30,0/0,0/1,0	ALLERGENO NON PRESENTI
ARABICA	g	41,5	1,8/0,0/0,0	ALLERGENO NON PRESENTI
ARROSO	g	742,0	0,0/0,0/0,0	LATTE
CACAO	g	430,0	19,8/17,0/1,0/0	ALLERGENO NON PRESENTI
CAMPANULA	g	255,0	0,0/0,0/0,0	LATTE
CAPRE	kg	0,4	0,0/0,0/0,0	ALLERGENO NON PRESENTI
CARAMELLO	g	371,0	0,0/0,0/0,0	ALLERGENO NON PRESENTI
CAROTI	g	43,0	0,0/0,0/0,0	ALLERGENO NON PRESENTI
CARVOFANO	g	30,3	1,8/1,0/0,0	ALLERGENO NON PRESENTI
CECI	g	168,6	8,9/12,0/2,8	ALLERGENO NON PRESENTI
CECILIATIONE INGREDIENTI	g	530,0	0,0/0,0/0,0	ALLERGENO NON PRESENTI
CEPOLA	g	42,5	11,0/0,0/1	ALLERGENO NON PRESENTI
CECCO CIOCA	kg	42,0	0,0/0,0/0,0	ALLERGENO NON PRESENTI
CECCO CIOCA 2500	g	0,4	0,0/0,0/0,0	ALLERGENO NON PRESENTI



Ristorante il Casale

Manager Ristorante

AGGIUNGI NUOVO INGREDIENTE

Nome ingrediente *

Unità di riferimento *

Valori nutrizionali

Carboidrati *	Proteine *	Grassi *

Allergeni

☐ Anziché soffice e soffici
 ☐ Molluschi

☐ Anacardi
 ☐ Pesce

☐ Cereali (glutine)
 ☐ Selenio

☐ Crostacei
 ☐ Senape

☐ Frutta a guscio
 ☐ Serrano

☐ Latte e derivati
 ☐ Sola

☐ Lupini
 ☐ Uova

Riepilogo nutrizionale

Calorie totali

0.0 kcal

Carboidrati

0.0 g

Proteine

0.0 g

Grassi

0.0 g

Allergeni presenti

Nessun allergene segnalato

SALVA

ANNULLA

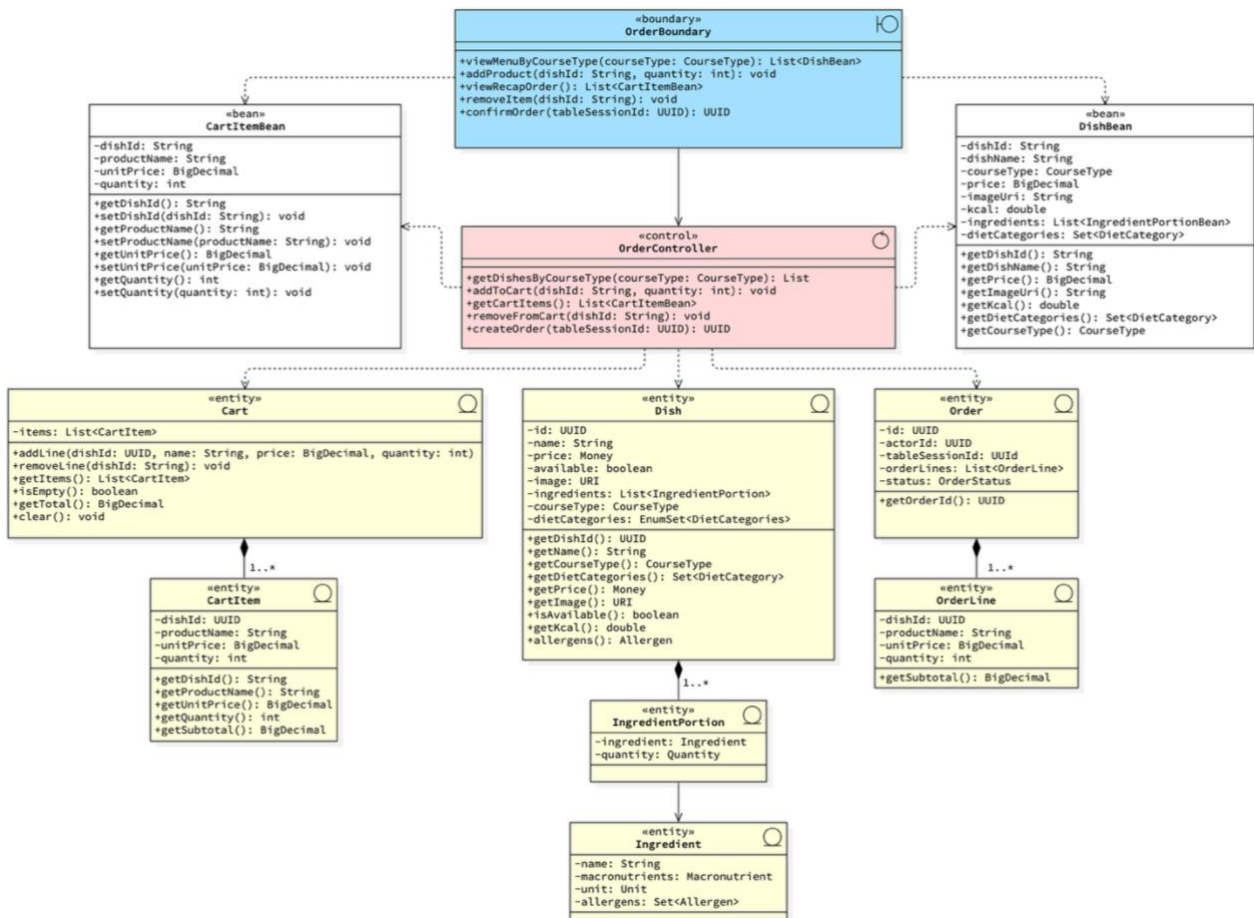
3. DESIGN

3.1 CLASS DIAGRAM

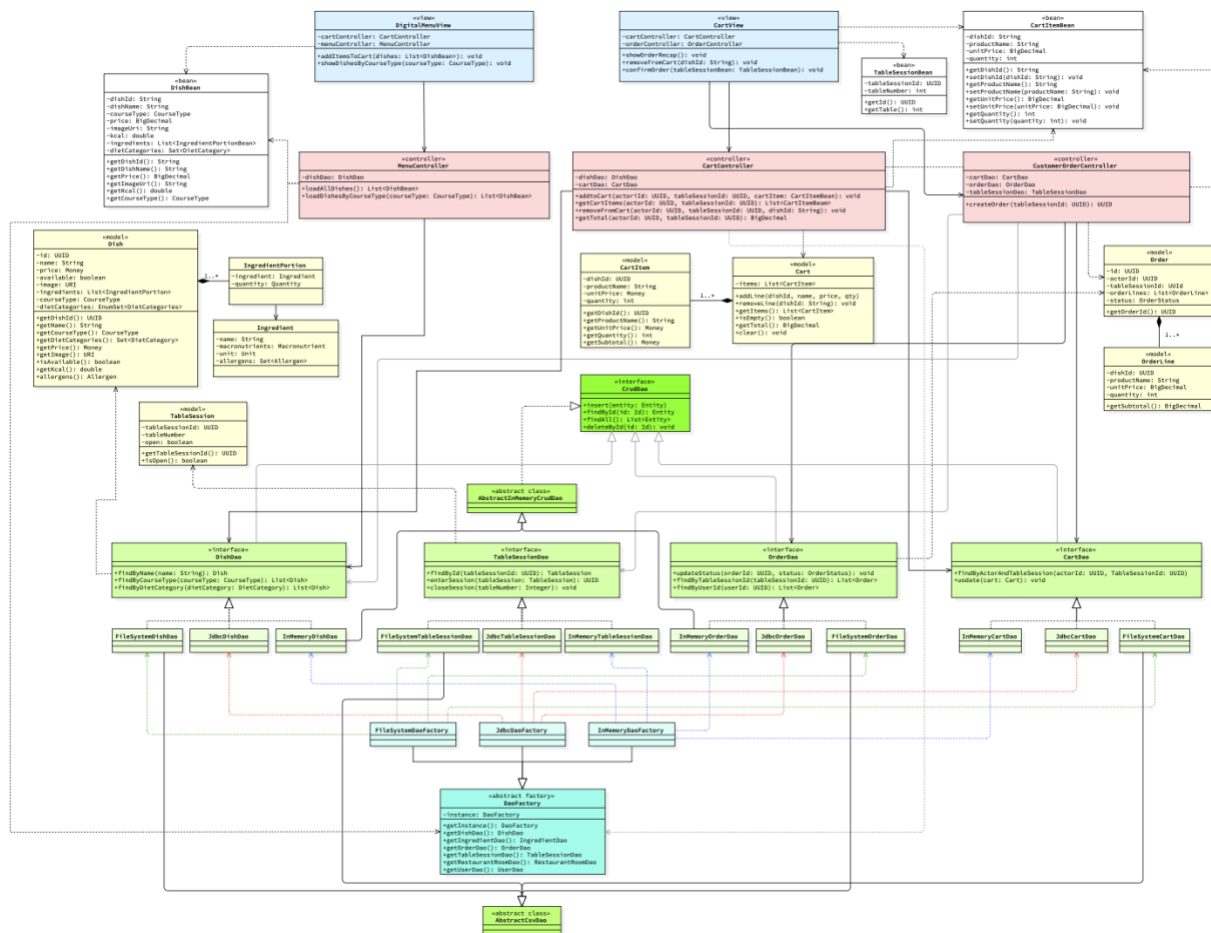
I diagrammi che seguono sono presenti nella cartella **deliverables** del progetto.

3.1.1 VOPC

VOPC relativo al caso d'uso "Ordina".



3.1.2 DESIGN LEVEL



NOTE

Il diagramma rappresenta una versione semplificata rispetto al codice implementato. Alcuni dettagli infrastrutturali e alcune dipendenze interne alle concrete factory, sono omessi per mantenere il diagramma leggibile.

3.2 DESIGN PATTERNS

Il progetto adotta specifiche soluzioni di progettazione con l'obiettivo di separare le responsabilità, ridurre l'accoppiamento tra componenti e rendere l'applicazione configurabile rispetto alle diverse modalità di esecuzione e persistenza.

In particolare, la gestione della persistenza e della fase di avvio è stata strutturata tramite l'impiego dei pattern **Abstract Factory**, **Singleton**, **Decorator** e **Simple Factory**.

3.2.1 ABSTRACT FACTORY

Abstract Factory per la selezione della famiglia di DAO

L'applicazione supporta più modalità di persistenza, selezionabili in fase di avvio (**FULL**, **DEMO**, **FILESYSTEM**).

Per gestire tale flessibilità è stato impiegato il pattern **Abstract Factory**, che permette di selezionare e utilizzare una famiglia coerente di **DAO** senza vincolare i client alle relative implementazioni concrete.

La classe **DaoFactory** rappresenta l'**Abstract Factory** e definisce il contratto comune per l'ottenimento dei DAO (es. `getDishDao()`, `getOrderDao()`, `getUserDao()`, ecc.).

Le classi **JdbcDaoFactory**, **InMemoryDaoFactory** e **FileSystemDaoFactory** rappresentano le Concrete Factory.

Ciascuna di esse è responsabile della creazione, configurazione e composizione dei DAO appartenenti alla propria famiglia di persistenza.

I DAO vengono esposti tramite le rispettive interfacce (**DishDao**, **IngredientDao**, **OrderDao**, ecc.), consentendo ai controller e agli altri client di dipendere dalle astrazioni anziché dalle implementazioni concrete.

La factory gestisce inoltre le dipendenze tra i DAO. Ad esempio, **JdbcDishDao** e **FileSystemDishDao** ricevono un riferimento a **IngredientDao** tramite costruttore, senza conoscere direttamente la relativa implementazione concreta.

In fase di startup viene invocato **DaoFactory.init(mode)**, così da determinare in modo univoco la famiglia di DAO da utilizzare durante l'esecuzione dell'applicazione.

In questo modo viene sfruttato il **polimorfismo**: il tipo del riferimento rimane **DaoFactory**, mentre l'implementazione concreta utilizzata viene determinata a runtime in funzione della modalità di persistenza selezionata.

3.2.2 SINGLETON

Singleton per la gestione delle risorse condivise

Il pattern **Singleton** è utilizzato per i componenti che, per responsabilità architetturale, devono essere disponibili come unica istanza condivisa durante l'esecuzione dell'applicazione.

In particolare, **DaoFactory** viene inizializzata una sola volta tramite **init(mode)** e successivamente recuperata mediante **getInstance()**.

L'unica istanza di **DaoFactory** mantiene stabile la famiglia di persistenza scelta durante la fase di bootstrap e permette ai client di ottenere i DAO attraverso un punto di accesso comune.

I singoli DAO concreti sono gestiti dalle rispettive **Concrete Factory**, che mantengono le istanze necessarie e le restituiscono attraverso le interfacce DAO.

JdbcConnectionManager adotta il pattern **Singleton** per centralizzare la gestione della connessione JDBC. L'istanza viene inizializzata tramite **init(url, user, password)** e successivamente recuperata tramite **getInstance()**.

La connessione viene creata in modo lazy solo quando richiesta, evitando che ciascun DAO gestisca autonomamente una propria connessione al database.

3.2.3 DECORATOR

Decorator per l'aggiunta della cache ai DAO

Per introdurre il **caching** senza modificare le implementazioni concrete dei DAO è stato utilizzato il pattern **Decorator**.

L'interfaccia **IngredientDao** rappresenta il Component. Nelle modalità di persistenza FULL e FILESYSTEM, rispettivamente **JdbcIngredientDao** e **FileSystemIngredientDao** rappresentano i Concrete Component ai quali viene aggiunta la responsabilità di caching.

La classe **CachedIngredientDao** implementa a sua volta l'interfaccia **IngredientDao** e mantiene internamente un riferimento a un altro **IngredientDao**, denominato delegate.

In questo modo **CachedIngredientDao** può aggiungere il comportamento di caching alle operazioni del DAO, delegando al componente sottostante l'accesso effettivo al meccanismo di persistenza quando il dato non è già disponibile in memoria. Le operazioni di inserimento e rimozione vengono inoltre propagate al DAO sottostante e utilizzate per mantenere coerente il contenuto della cache.

Per evitare la duplicazione della logica di caching è stata inoltre introdotta la classe astratta generica **AbstractCachedDao<T, I>**, che incapsula le operazioni comuni di gestione della cache, come recupero, inserimento, rimozione e svuotamento degli elementi memorizzati.

CachedIngredientDao estende tale classe e implementa **IngredientDao**, aggiungendo il comportamento di caching senza modificare le implementazioni concrete di persistenza. La classe astratta costituisce quindi un supporto riutilizzabile all'implementazione del Decorator e permette, qualora necessario, di estendere in futuro lo stesso meccanismo ad altri DAO.

La composizione degli oggetti viene effettuata dalle relative Concrete Factory, che costruiscono il DAO concreto e lo decorano mediante **CachedIngredientDao**.

Il client continua quindi a lavorare esclusivamente tramite l'interfaccia **IngredientDao**, senza conoscere né il meccanismo di caching né l'implementazione concreta utilizzata per la persistenza.

Nella modalità DEMO non viene applicato il Decorator, poiché **InMemoryIngredientDao** utilizza direttamente una struttura in memoria come meccanismo di persistenza.

3.2.4 FACTORY

Simple Factory per selezionare l'interfaccia grafica

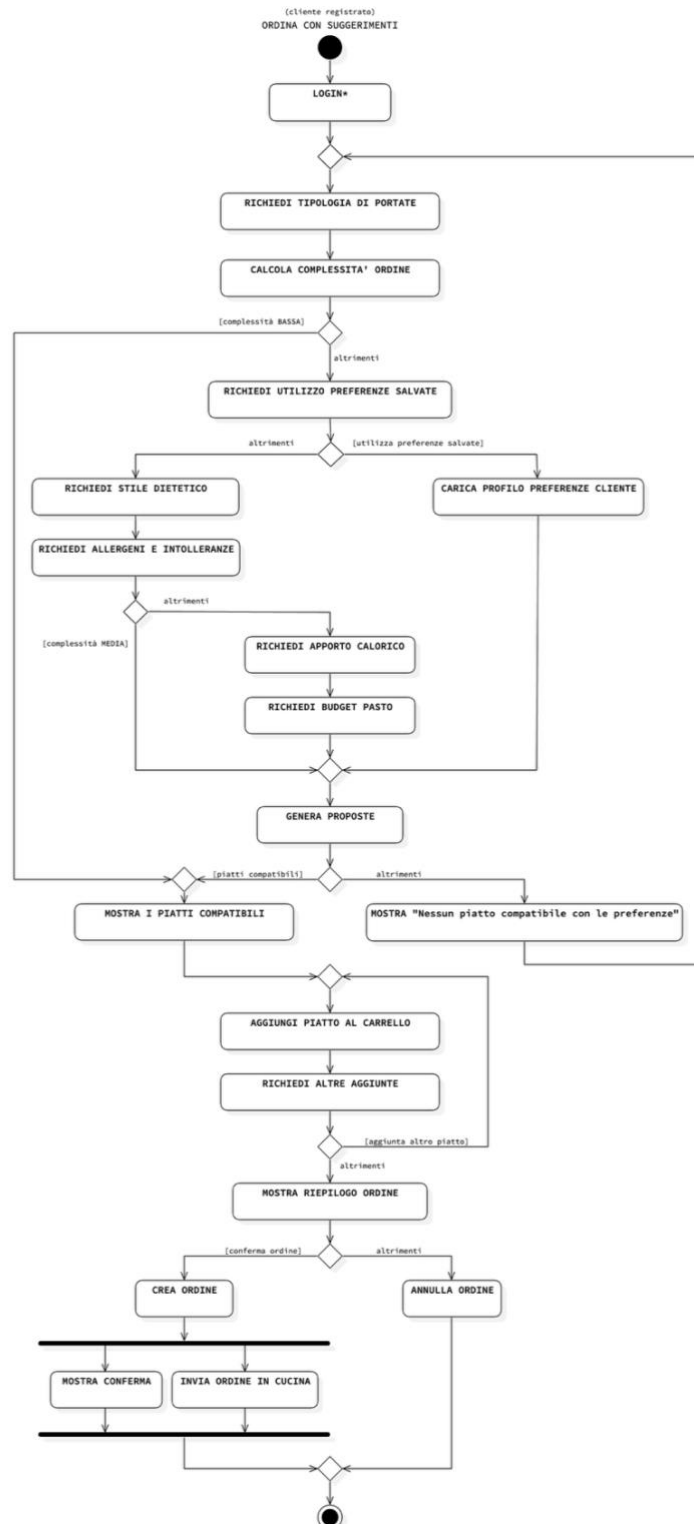
La selezione e l'avvio dell'interfaccia utente sono gestiti tramite **InterfaceFactory**, che crea l'istanza di **InterfaceBase** coerente con la modalità UI selezionata (**UiMode.CLI** oppure **UiMode.GUI**).

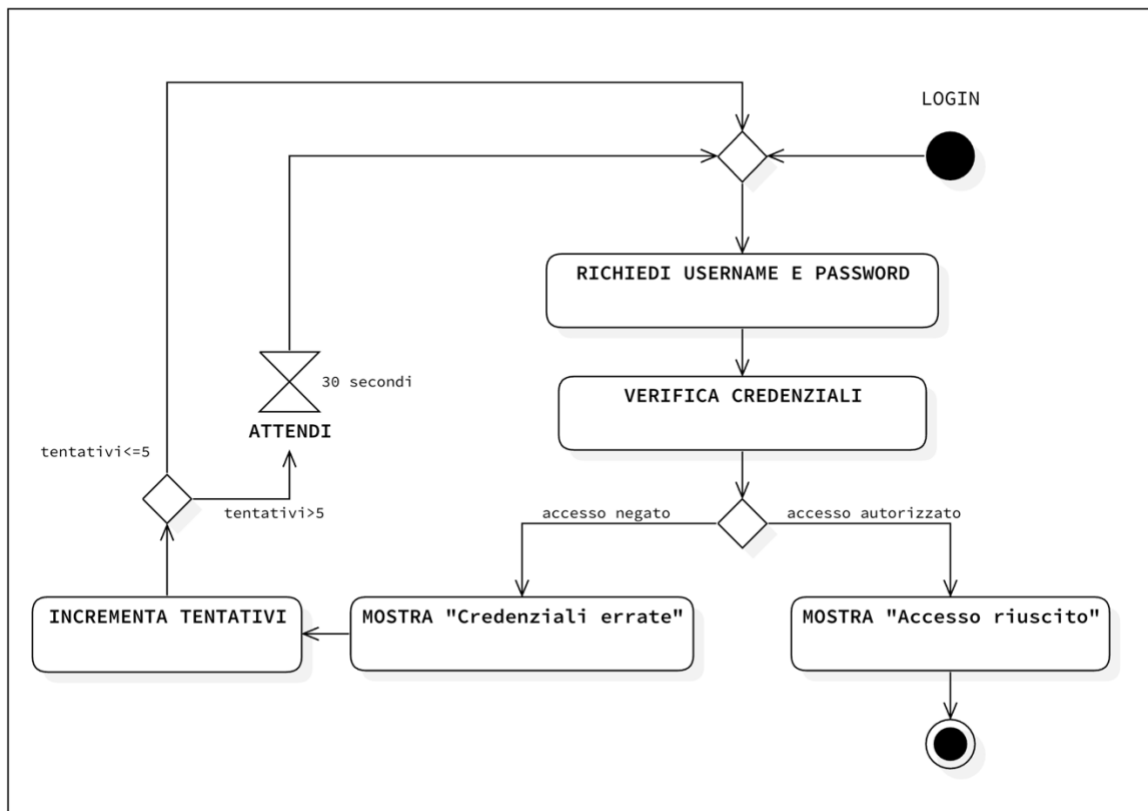
Il client utilizza il riferimento attraverso il tipo **InterfaceBase**, mentre l'oggetto concreto restituito dalla factory può essere **CliInterface** oppure **GuiInterface**.

In questo modo la scelta dell'implementazione concreta dell'interfaccia utente rimane concentrata nella factory e il resto dell'applicazione può utilizzare il polimorfismo senza dipendere direttamente dalle classi concrete della UI.

3.3 ACTIVITY DIAGRAM

Per contestualizzare il flusso del diagramma, la complessità dell'ordine viene classificata in tre livelli (bassa, media, alta) in funzione del numero e del tipo di portate selezionate. Tale classificazione è utilizzata per decidere quanti passaggi del flusso sono necessari.

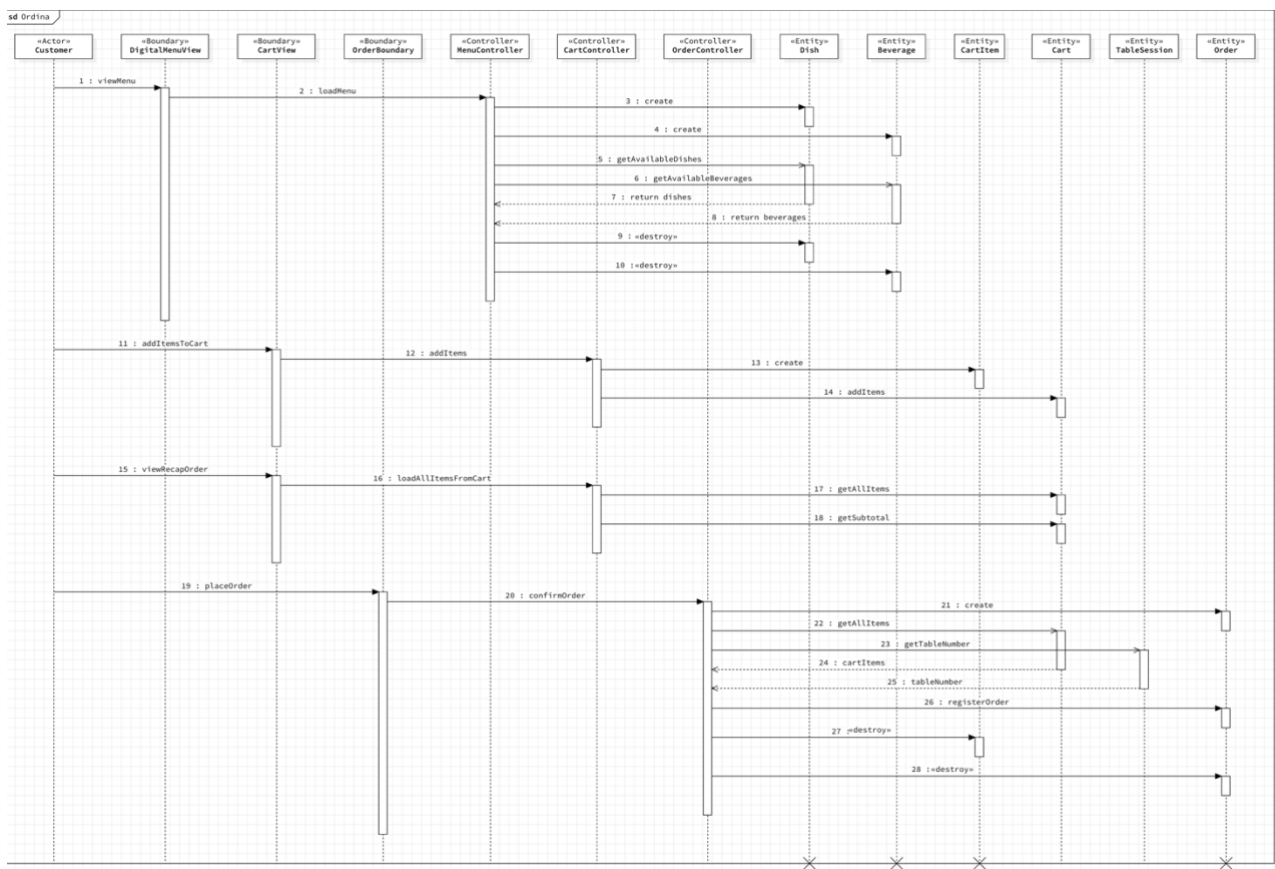




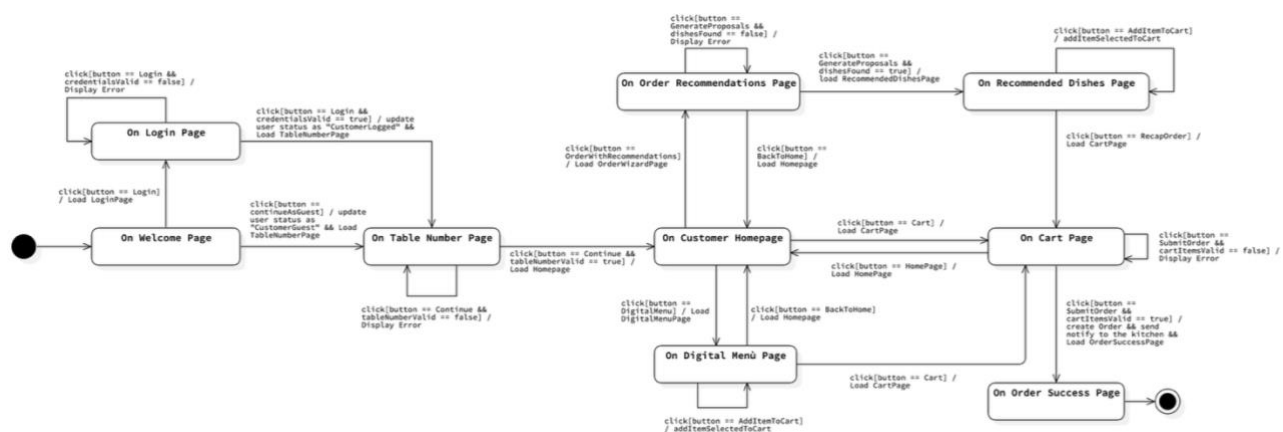
NOTE

Il caso d'uso "Ordina con suggerimenti" può essere eseguito anche da un utente ospite, l'unico limite rispetto a un utente registrato è che l'ospite non dispone di un profilo preferenze persistente, quindi, non può richiamare preferenze salvate né riutilizzarle in sessioni successive.

3.4 SEQUENCE DIAGRAM



3.5 STATE DIAGRAM



4. TESTING

I test del progetto si trovano nella cartella `src/test/java/it/foodmood`.

Sono stati scritti con **JUnit 5** e utilizzano le asserzioni (ad esempio `assertEquals`, `assertTrue`, `assertFalse`, `assertNotNull`, `assertThrows` ...) per verificare che i metodi producano i risultati attesi oppure generino le eccezioni corrette nei casi di errore.

I test coprono sia una parte di logica applicativa e sia una parte di logica di dominio.

È possibile eseguire tutti i test da terminale tramite il comando `mvn test`.

Il file **LoginControllerTest** controlla il comportamento del login. Vengono provati:

- **Login corretto:** non deve lanciare eccezioni, la sessione deve esistere e l'utente in sessione deve avere l'e-mail attesa.
- **Credenziali errate:** deve lanciare `AuthenticationException` e non creare la sessione.

Il file **DishTest** verifica la creazione di un piatto a partire dai parametri, vengono creati ingredienti con macronutrienti, unità di misura e allergeni, e poi viene costruita una "Carbonara". Il test controlla che nome e tipo portata siano corretti, che gli allergeni siano rilevati correttamente, che le kcal siano maggiori di zero, che il piatto risulti disponibile e che l'ID venga assegnato.

Il file **UserTest** verifica la creazione di utenti con ruoli diversi (Customer, Waiter, Manager) e il funzionamento delle credenziali. Per ogni utente controlla dati anagrafici, e-mail, ruolo e verifica che l'hash della password funzioni.

5. CODE

5.1 GITHUB

GitHub: <https://github.com/montigabriele/Progetto-Ingegneria-del-Software>

5.2 SONARCLOUD

SonarCloud: <https://sonarcloud.io/organizations/foodmood/projects>

6. VIDEO

6.1 YOUTUBE

YouTube: https://youtu.be/c_-QABC37fs